MCP vs API

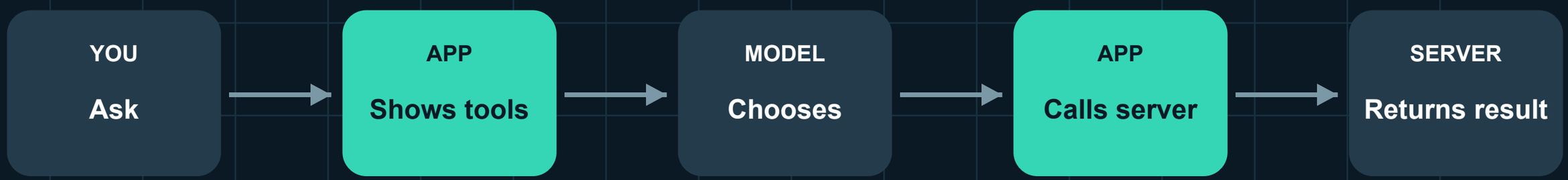
A clear interview guide

What MCP is. How tool calls work. When to use each.

THE DISTINCTION TO REMEMBER

The model chooses the tool.
The app makes the call.

MCP discovers tools. An API is how code talks to a service.



The difference • Tool flow • Design choices • Cost • Safety

A practical guide for engineers and AI builders preparing for interviews.

01 / THE INTERVIEW ANSWER

What is the difference between MCP and an API?

Define each, then explain who chooses and who executes.

API

Code talks to a service. The developer chooses an endpoint and writes the steps.

MCP

An AI app discovers a service's tools at runtime and gives the model a menu.

An MCP server usually sits in front of a normal API. It does not replace the API. Function calling still matters: the model returns a tool name and inputs; the app runs it. MCP standardizes where the tool list comes from and how the call reaches the service.

How a tool call works

1. App asks the server for tools: tools/list.
2. Server returns tool names, descriptions, and input schemas.
3. App gives the tool menu and the user's question to the model.
4. Model chooses a tool and supplies inputs. It does not run the tool.
5. App sends tools/call. Server does the work, often through an API, and returns the result.

Roles: The host is the AI app. Its client talks to one server. The server can expose tools (callable functions), resources (file-like data), and prompts (ready-made templates). A result may contain text, images, audio, links, or structured content that follows an output schema.

Example tools/call message

```
{
  "jsonrpc": "2.0", "id": 2,
  "method": "tools/call",
  "params": { "name": "get_weather",
   "arguments": { "location": "New York" }
  }
}
```

02 / MAKE THE DESIGN CHOICE

When should you use MCP or call the API directly?

Choose based on who should decide the next step.

Question	API	MCP
Built for	A developer writing code.	An AI app and its model.
How capabilities are found	Read docs and write code for each endpoint.	Ask the server at runtime with tools/list.
Who chooses the next call	Your code.	The model, from a tool menu.
Best for	Fixed steps, such as taking a payment.	Open-ended tasks where the model must choose.

Order lookup and refunds

MCP

Expose order lookup and policy search as tools. The model chooses between them based on the customer’s request.

Plain code

Keep refunds in a normal function. Check the amount limit in code and require human approval for large refunds.

Rule of thumb: If you can write the steps down in advance, write code. If the model has to decide, give it tools. A product can use both.

03 / TECHNICAL DETAILS

How do transport, errors, versions, and auth work?

Use the terms precisely; explain what changes in practice.

Transport

stdio: The client starts a same-machine server as a subprocess. Protocol messages go on stdout; logs go to stderr.

Streamable HTTP (2026-07-28): Each client request uses POST; the server replies with JSON or request-scoped SSE. This revision removed the GET stream. Check the Origin header; bind local servers to 127.0.0.1.

Errors

Unknown tool or malformed request: protocol error.

Tool failure, such as a failed API call or bad input: normal result with `isError: true`, so the model can read it and try again.

Versions and messages

The supplied notes identify `2026-07-28` as the current revision. It is stateless and supports `server/discover`. Earlier revisions through `2025-11-25` use an `initialize` handshake and can use sessions. Streamable HTTP revisions `2025-03-26` through `2025-11-25` also had a GET SSE stream. Check which revision your SDK targets; they differ. MCP messages use JSON-RPC encoded as UTF-8.

Tool calling without MCP

In Anthropic's API, the model returns a `tool_use` block, code runs the tool, and the app sends back a `tool_result`. MCP works alongside that loop and standardizes tool discovery and routing.

Auth and the Anthropic connector

Authorization is optional. HTTP authorization follows an OAuth 2.1-based flow; stdio servers should read credentials from the environment. Anthropic's beta Messages API connector reaches remote MCP servers over HTTP and supports tool calls only; local stdio servers are not supported.

04 / COST AND SAFETY

What does MCP cost, and how do you keep it safe?

Tool definitions use context. Tool descriptions deserve trust checks.

Context cost

Most clients load every tool definition into context up front. Many tools can use tokens before the user asks anything.

150,000	→	2,000
tokens		tokens

Anthropic reported this reduction in one example by having the model write code against tools and load only what it needs. It is not a universal result. The spec also asks servers to list tools in the same order each time, which can help prompt-cache hits.

Connect only the servers you need.

Trust and safety

Tool descriptions go to the model. A malicious description can hide instructions the user never sees. The notes cite an April 2025 Invariant Labs example that instructed an agent to read an SSH key and send it out. A server can change a description after approval, or affect how the model uses another server’s tool.

Four safeguards

1. Connect only to servers you trust; pin versions.
2. Show tool inputs; let users deny sensitive calls.
3. Request the smallest OAuth scopes that work.
4. Put action limits in code, such as a refund cap.

The spec says clients should show inputs and allow a human to deny calls. Treat tool annotations as untrusted unless the server is trusted.

05 / INTERVIEW REVIEW

What should you avoid saying in an interview?

Use these corrections, then keep the one-sentence answer ready.

“MCP replaces APIs.”	Correction An MCP server usually calls an API.
“The model calls the server.”	Correction The app sends the call. The model selects the tool and supplies inputs.
“MCP is an Anthropic product.”	Correction Anthropic created it, but it is an open standard. The notes list ChatGPT, VS Code, and Cursor as supporters.
“Use MCP for everything.”	Correction Fixed steps, such as payments, belong in ordinary code.

Quick answer: An API is how code calls a service. MCP gives an AI app a standard way to discover tools. The model chooses; the app executes. Use MCP for model-directed choices and direct code for fixed, sensitive steps.

Further reading

What is MCP

Introduction to MCP

Build an MCP server

Tools spec

Transports spec

Authorization

Versioning

Anthropic tool use

MCP connector

Code execution with MCP

Tool poisoning attacks

MCP donation announcement

Terms to review: host, client, server; tools/list and tools/call; JSON-RPC; stdio and Streamable HTTP; OAuth; tool poisoning; code execution with MCP.